POS Token and Edge Relay - Full Development Context (AI Handoff)

1. Purpose and Audience

This document is the authoritative handoff for all development completed so far around POS token flow and Edge Relay integration in this repository. It is intentionally detailed so another AI agent or engineer can resume work with full technical and operational context.

2. Repository / Branch Context

• Repository: webdev3pj/POS-Awesome-pj

• Base branch where investigation started: fix/app-install

• Main implementation sequence used:

- cline-codex-v1

- cline-codex-v2

- kilo-codex-v1

• Current branch for this handoff document: kilo-codex-v1

3. Product Requirement Summary (as executed)

1. Implement token + edge relay workflow only for selected POS Profiles.

2. If feature is disabled on a profile, existing behavior must remain unchanged.

3. Build a practical local Edge Relay service for OptiPlex-style deployment.

4. Provide visible diagnostics in POS so operators can verify relay and cloud connectivity.

5. Keep secrets/runtime artifacts out of GitHub.

4. Commits and Capability Map

4.1 Commit 7450451

Message: feat: add POS-profile-gated relay workflow state for token/pick/dispatch

Key files:

• posawesome/hooks.py

• posawesome/posawesome/api/posapp.py

• posawesome/posawesome/doctype/pos_relay_workflow_state/*

• posawesome/public/js/posapp/components/pos/Invoice.vue

• posawesome/public/js/posapp/components/pos/Payments.vue

What it added:

• Relay workflow state model and logic (token, picking, dispatch)

• Profile-level gating using existing POS Profile checkbox custom_have_token

• Invoice lifecycle hooks to create/update workflow state

• Basic POS UI relay cues and submit-time queue message

4.2 Commit c53e4db

Message: feat(relay): add windows-friendly edge relay app with setup UI and realtime queue dashboard

Key files:

• relay/relay/app.py

• relay/relay/storage.py

• relay/relay/sync_worker.py

• relay/relay/windows_setup.py

• relay/relay/templates/*

• relay/start_relay.bat

• relay/README.md

• relay/SETUP_CHECKLIST_OPTIPLEX.md

What it added:

• Flask-based Edge Relay service

• Setup page, queue dashboard, health endpoint, metrics endpoint

• SQLite-backed event queue and sync worker

• Windows bootstrap automation helpers

4.3 Commit 0ec4e96

Message: feat(relay): one-click installer with auto python/deps and self-test

Key files:

• relay/start_relay.bat

• relay/relay/selftest.py

• docs updates in relay/README.md and relay/SETUP_CHECKLIST_OPTIPLEX.md

What it added:

• One-click startup path

• Dependency bootstrap behavior

• Self-test routine for quick validation

4.4 Commit b92129d

Message: chore(security): ignore relay local data and runtime artifacts

Key file:

• .gitignore

What it added:

• Ignore runtime/secret-prone relay artifacts (while preserving .gitkeep)

4.5 Commit 6f0b165

Message: feat(relay): add profile-gated edge relay connectivity status in POS

Key files:

• posawesome/posawesome/api/posapp.py

• posawesome/public/js/posapp/components/Navbar.vue

What it added:

• Backend connectivity endpoint to test relay reachability

• POS top-bar relay status chip with periodic polling

4.6 Commit 460e961

Message: feat(relay): per-profile edge relay URL and richer POS diagnostics

Key files:

• posawesome/fixtures/custom_field.json

• posawesome/fixtures/property_setter.json

• posawesome/posawesome/api/posapp.py

• posawesome/public/js/posapp/components/Navbar.vue

What it added:

• New POS Profile field custom_edge_relay_url

• Profile-specific relay URL support (with site fallback)

• Expanded relay diagnostics (source URL, status class, health URL, hints)

4.7 Commit 760c7c9

Message: feat(ui): add cloud internet connectivity status chip with diagnostics

Key file:

• posawesome/public/js/posapp/components/Navbar.vue

What it added:

• Additional top-bar cloud/internet connectivity status chip

• Browser online/offline + cloud reachability probe

• Latency, HTTP status, checked-at and message diagnostics

5. Architecture and Flow Details

5.1 POS Profile Gating

• Primary gating field: custom_have_token (existing field)

• New profile relay endpoint field: custom_edge_relay_url

• Principle: all relay/token logic is disabled when custom_have_token is off

5.2 Workflow State Model

• DocType: POS Relay Workflow State

• Core state dimensions:

- token_status (Draft/Paid/Expired/Abandoned)

- picking_status (Not Started/In Progress/Picked/Exception)

- dispatch_status (Pending/Released/On Hold)

• State updates happen during invoice save/submit and via explicit relay methods

5.3 Edge Relay Service Responsibilities

• Receive local events from external systems/devices (token/pick/release)

• Queue events durably (SQLite)

• Sync to Frappe cloud via API tokens

• Provide operational visibility:

- /health

- /api/metrics

- /api/queue

5.4 POS UI Status Layer

In Navbar:

1) Relay status chip (profile-scoped):

- Online / Not Configured / Timeout / Connection Error / HTTP Error / Offline

- Opens diagnostics card with helpful details and hints

2) Cloud connectivity chip:

- Internet Offline / Cloud Online / Cloud Unreachable

- Uses browser online signal + request to frappe auth endpoint

6. Concrete Files to Inspect First (for any future work)

1. posawesome/posawesome/api/posapp.py

2. posawesome/public/js/posapp/components/Navbar.vue

3. posawesome/fixtures/custom_field.json

4. posawesome/fixtures/property_setter.json

5. relay/relay/app.py

6. relay/relay/sync_worker.py

7. relay/relay/storage.py

8. relay/start_relay.bat

7. Operational Checks Already Performed

• Relay health endpoint reachable locally (HTTP 200, ok true)

• Relay queue endpoints accepted and reflected queued events

• Token authentication to Frappe cloud validated using provided API key/secret

• During one test window, cloud lacked new methods (indicating deploy mismatch), confirming deployment/migration is required for end-to-end

• Python compile checks executed for modified backend file

• Fixture JSON parse checks succeeded

8. Known Constraints and Caveats

1. Cloud site must deploy this branch and run migrate/restart before new methods appear.

2. Local relay may run fine even if cloud API methods are not yet deployed.

3. Untracked local artifacts observed in workspace:

- POS_Relay_End_to_End_Plan_with_Workflow_Exceptions_and_Edge_Cases.pdf

- attendance.db

These should not be committed unless explicitly needed.

9. Pending Work (Actionable)

9.1 Deployment Alignment (highest priority)

• Deploy branch kilo-codex-v1 to cloud app

• Run bench migrate and restart services

• Verify methods resolve:

- get_relay_connectivity_status

- get_relay_workflow_state

- update_relay_picking_status

- release_relay_dispatch

9.2 End-to-End UAT

For one enabled profile:

1. Set custom_have_token = 1

2. Set custom_edge_relay_url to target edge host

3. Open POS and verify both status chips

4. Create and submit invoice

5. Validate token/pick/dispatch transitions through relay and cloud

6. Confirm queue/error handling paths and operator diagnostics

9.3 Hardening Backlog

• Add robust retry policy/backoff for sync worker

• Add richer failure telemetry and audit records

• Improve permission boundary and security review for relay endpoints

• Add localization pass for all diagnostic labels/messages

• Add formal operator runbook with screenshots

10. Rollback and Safety Notes

• Feature is profile-gated; turning off custom_have_token should disable workflow behavior for that profile.

• If relay URL invalid, diagnostics should indicate not configured or connection error without changing non-relay POS functionality.

• Keep API keys only in runtime config; do not commit secrets into tracked files.

11. Final State Snapshot

• Branch: kilo-codex-v1

• Latest commit expected in remote before this document work: 760c7c9

• Includes:

- profile-specific relay URL setting

- detailed relay diagnostics

- cloud internet/connectivity diagnostics

12. Quick Resume Checklist for Next AI

1. Pull latest kilo-codex-v1.

2. Verify deployed cloud app has new methods.

3. Validate POS Profile field visibility and saved URL.

4. Validate relay + cloud chips in POS header.

5. Execute full invoice -> relay -> workflow state UAT.

6. Resolve any runtime errors with reproducible logs and commit minimal scoped fixes.